



Amor de Patas – Estilização e Layouts

Desenvolvimento FrontEnd - Experiência Prática 2



2 Identificação do aluno

IS

**Isabel Cristina dos Santos Tombini CRUZ_EAD_Cst em Análise e Desenvolvimento de
Sistemas_1A_20262**



Variáveis do Design System

O Design System do projeto foi implementado por meio de variáveis CSS nativas, conhecidas como custom properties, centralizadas no arquivo `assets/css/styles.css` e reutilizadas nas demais folhas de estilo, como `index.css`, `projetos.css` e `cadastro.css`. Essa organização permite manter uma identidade visual consistente entre as diferentes páginas do projeto, além de facilitar futuras alterações e a manutenção dos estilos. A paleta de cores é composta por dez variáveis, distribuídas entre cores primárias, secundárias e neutras. Como cores primárias, foram definidos o --azul-noite: #1B2A40, utilizado como cor de marca em elementos como cabeçalho, rodapé e títulos, e o --azul-medio: #34506F, empregado principalmente em variações, gradientes e estados de hover dos links. Para as cores secundárias e de destaque, foram utilizados o --mel: #E3A23B, destinado principalmente às ações, como botões e outros elementos de destaque, e o --mel-hover: #EEB35E, utilizado nos estados de hover e foco. A composição é complementada pelas cores neutras --creme: #FBF6EC, utilizada como fundo principal; --branco: #FFFFFF, presente em cartões e formulários; --texto: #23262B, para os textos principais; --texto-suave: #6B6A5E, para textos auxiliares e legendas; --texto-claro: #C9D6E3, utilizado sobre fundos escuros; e --linha: #E1D8C4, aplicado em bordas e divisores. A tipografia foi organizada em oito patamares hierárquicos, permitindo diferenciar os conteúdos de acordo com sua importância visual. A variável --text-xs, com 0.8rem, é destinada às legendas; --text-sm, com 0.9rem, aos labels e elementos de navegação; --text-base, com 1rem, ao corpo dos textos; e --text-md, com 1.1rem, aos parágrafos de destaque. Para elementos de maior hierarquia, foram definidos --text-lg, com 1.25rem, para subtítulos h3 e legend; --text-xl, com 1.5rem, para logo e elementos de ênfase; e --text-2xl, com 2rem, para títulos de seção h2. Por fim, o --text-display, definido como `clamp(2.2rem, 5vw, 3.4rem)`, é utilizado no título principal h1, permitindo que seu tamanho se adapte de maneira responsiva à largura da viewport. Os espaçamentos seguem uma escala modular baseada em 4px, iniciando em --space-1, com 4px, e chegando a --space-12, com 88px. A escala utiliza os valores 4, 8, 12, 16, 20, 24, 32, 40, 48, 64, 72 e 88px, aplicados em paddings, margens e gaps de grid ao longo do projeto. A adoção dessa padronização evita o uso de valores arbitrários e contribui para manter um ritmo visual consistente entre os diferentes componentes e seções da interface. As decisões visuais também consideram a acessibilidade e o perfil diversificado do público de uma organização do terceiro setor. A paleta prioriza combinações de alto contraste entre o azul-noite e os fundos creme ou branco, buscando favorecer a legibilidade para diferentes usuários, desde jovens voluntários até doadores mais velhos. A cor mel foi reservada principalmente para ações, como botões e links ativos, facilitando a identificação dos elementos interativos sem competir visualmente com o conteúdo principal. Da mesma forma, a escala tipográfica mantém uma hierarquia clara e proporções adequadas para diferentes tamanhos de tela. Por fim, a utilização de uma escala modular de espaçamento proporciona maior organização e consistência à interface, além de facilitar a manutenção futura do código e preservar a identidade visual em todas as páginas da plataforma.

Código Grid

O sistema de Grid do projeto foi implementado por meio de uma classe utilitária de 12 colunas, definida no arquivo `assets/css/styles.css`. A classe `.grid-12` utiliza `display: grid`, juntamente com `grid-template-columns: repeat(12, 1fr)`, dividindo o espaço disponível em doze colunas de mesma proporção. O espaçamento entre elas é definido por `gap: var(--space-6)`, seguindo a escala de espaçamentos estabelecida no Design System. Essa estrutura é complementada por classes reutilizáveis, como `.col-3`, `.col-4`, `.col-5`, `.col-6`, `.col-7`, `.col-8` e `.col-12`, que utilizam `grid-column: span N` para determinar quantas das doze colunas cada elemento deve ocupar. A estrutura foi aplicada aos diferentes layouts de acordo com as necessidades de cada página. Nos containers `.grid-animais` e `.formas-ajuda`, presentes em `index.html`, e `.grid-projetos`, em `projetos.html`, cada `article` ocupa quatro colunas, formando três elementos de mesma largura na distribuição $4+4+4=12$. Nas seções `.sobre`, `.wrap` e `.contato`, o conteúdo principal ocupa sete colunas e o bloco lateral, destinado às estatísticas ou ao aside, ocupa cinco, formando a distribuição $7+5=12$ e atribuindo maior espaço visual ao conteúdo textual. Já em `.linha-campos`, utilizada em `cadastro.html`, cada campo ocupa seis colunas, permitindo a apresentação dos campos em pares, na proporção $6+6=12$. Para tornar essa estrutura responsiva, foram definidos cinco breakpoints: 480px, 640px, 768px, 1024px e 1440px. Em telas muito estreitas, os elementos passam a ocupar `span 12`, formando uma única coluna. Nas dimensões intermediárias, podem ocupar `span 6`, formando duas colunas, enquanto telas maiores utilizam novamente a estrutura completa de 12 colunas. Em telas panorâmicas, a partir de 1440px, os espaços entre as colunas são ampliados para aproveitar melhor a área disponível. As media queries baseadas em `max-width` foram organizadas em ordem decrescente, de 1024px até 480px. Essa organização considera que diferentes condições de `max-width` podem ser atendidas simultaneamente em uma mesma largura de tela. Dessa forma, quando regras possuem a mesma especificidade, a ordem em que aparecem no CSS contribui para determinar qual declaração prevalecerá, permitindo que os ajustes destinados às telas menores sejam aplicados corretamente. Assim, o Grid de 12 colunas mantém uma estrutura visual consistente, mas adapta a distribuição dos elementos conforme o espaço disponível em cada dispositivo.

Breakpoints e estratégia adotada

Os cinco breakpoints do projeto — 480px, 640px, 768px, 1024px e 1440px — estão distribuídos entre os arquivos `styles.css`, `index.css`, `projetos.css` e `cadastro.css`. Em 480px (celulares pequenos), o formulário empilha em 1 coluna e o logotipo reduz. Em 640px (celulares grandes), os cards de animais e projetos passam a 1 coluna. Em 768px (tablets retrato), os cards passam a 2 colunas e o menu se empilha verticalmente. Em 1024px (tablets paisagem/notebooks), missão e contato empilham, enquanto os cards retornam a 3 colunas. Em 1440px (desktops panorâmicos), os espaçamentos entre colunas aumentam e o container ganha mais largura. A estratégia foi organizar as media queries em ordem decrescente de largura, do breakpoint mais amplo ao mais estreito, garantindo que, quando condições de max-width coincidam numa mesma tela, a ordem de declaração no CSS assegure que os ajustes corretos prevaleçam em telas estreitas.



Módulos de componentes da *Flexbox*

- `header .wrap`: `display: flex; align-items: center; justify-content: space-between;` — alinha verticalmente o logotipo e o menu de navegação no cabeçalho, distribuindo-os nas extremidades opostas do contêiner.
- `nav ul`: `display: flex; flex-wrap: wrap; gap: var(--space-6);` — organiza os links de navegação em linha e permite a quebra para uma nova linha quando o espaço disponível diminui, evitando a sobreposição do conteúdo.
- `.grid-animais article`: `display: flex; flex-direction: column;`, com `flex-grow: 1`; aplicado ao parágrafo — organiza verticalmente os elementos internos do card e permite que o texto ocupe o espaço restante, contribuindo para manter o alinhamento visual entre os cards de uma mesma linha.
- `.formas-ajuda article`: `display: flex; flex-direction: column;`, com `flex-grow: 1`; aplicado ao parágrafo — organiza verticalmente o conteúdo dos blocos “Adote”, “Seja voluntário” e “Faça uma doação”, mantendo uma distribuição interna consistente.
- `.grid-projetos article`: `display: flex; flex-direction: column;`, com `flex-grow: 1`; aplicado ao parágrafo — mantém a organização vertical e o alinhamento interno dos cards de projetos sociais, mesmo quando as descrições apresentam comprimentos diferentes.



Processo de navegação responsiva

A navegação do projeto foi desenvolvida para se adaptar aos diferentes tamanhos de tela, mantendo a organização e a facilidade de uso tanto em computadores quanto em dispositivos móveis. Para isso, a transformação do menu horizontal em um sistema responsivo com dropdown e menu hambúrguer foi realizada por meio da combinação de seletores CSS, pseudo-classes, media queries e um pequeno trecho de JavaScript responsável pelo controle dos estados de abertura e fechamento por clique ou toque. Em telas maiores, o menu principal é apresentado horizontalmente utilizando `display: flex` no seletor `nav ul`. Os itens que possuem submenu recebem a classe `.tem-dropdown` com `position: relative`, que funciona como referência para o posicionamento do conteúdo secundário. O submenu `.dropdown-menu` permanece inicialmente oculto com `display: none` e pode ser exibido de diferentes maneiras. O seletor `.tem-dropdown:hover .dropdown-menu` permite sua abertura ao passar o mouse, enquanto `.tem-dropdown:focus-within .dropdown-menu` possibilita a interação durante a navegação por foco. A classe `.tem-dropdown.aberto`, por sua vez, permite controlar a abertura por clique com auxílio do JavaScript. Para telas com largura mínima de 769px, o submenu utiliza `position: absolute` e `top: 100%`, sendo apresentado abaixo do item correspondente sem deslocar o restante do conteúdo da página. Nas telas menores, a navegação é modificada pelo breakpoint `max-width: 768px`. O botão `.menu-toggle`, que permanece oculto com `display: none` nas telas maiores, passa a ser exibido com `display: block`. Ao mesmo tempo, o elemento `nav` fica inicialmente oculto e é apresentado quando recebe a classe `.aberto`. Os itens de navegação deixam a organização horizontal e passam a utilizar `flex-direction: column`. O cabeçalho também recebe `flex-wrap: wrap`, enquanto o menu utiliza `width: 100%` e `order: 3`, permitindo que a navegação seja posicionada em uma nova linha abaixo do cabeçalho quando aberta. O ícone do menu hambúrguer também apresenta uma resposta visual ao usuário. Ele é construído com o elemento principal e os pseudo-elementos `::before` e `::after`. Quando o atributo `aria-expanded` assume o valor `true`, são aplicadas transformações com `rotate(45deg)` e `rotate(-45deg)`, formando visualmente um "X". A propriedade `transition: transform 0.2s ease` torna essa alteração mais suave. O JavaScript complementa a implementação controlando os estados de interação. Ao clicar no botão hambúrguer, o atributo `aria-expanded` e a classe `.aberto` são alternados, permitindo que o CSS determine a aparência correspondente. A mesma lógica é aplicada aos dropdowns, que também podem ser fechados ao clicar fora da área do menu ou pressionar a tecla Esc. Dessa forma, a solução mantém o CSS responsável pela apresentação visual e utiliza o JavaScript para controlar os estados necessários à interação, proporcionando uma navegação adaptável a mouse, toque e teclado e preservando a clareza da interface em diferentes tamanhos de tela.



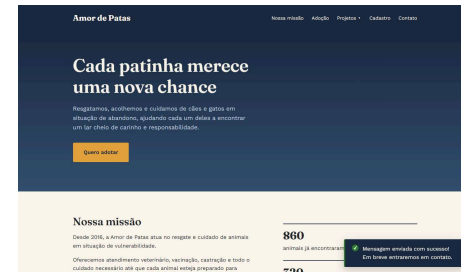
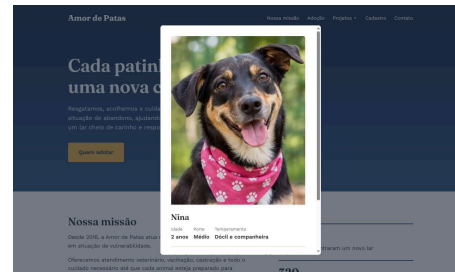
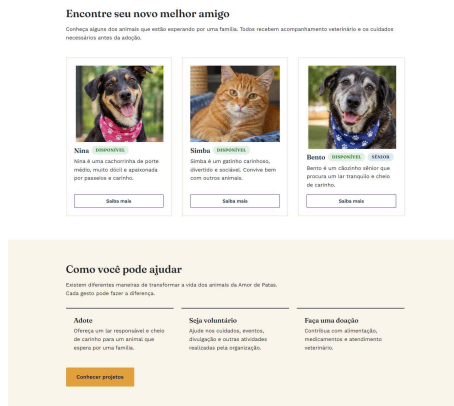
Encadeamento lógico dos estados visuais

Os estados interativos dos botões e a sinalização visual dos formulários foram tratados através do encadeamento de pseudo-classes nativas do CSS, combinadas com propriedades de cor, sombra, borda e transição, sem depender de JavaScript para a aparência. Nos botões (.btn e .btn-secundario), o estado padrão define background-color, cursor: pointer e uma transition: background-color 0.2s ease, box-shadow 0.2s ease, transform 0.1s ease, que serve de base para todas as mutações seguintes. No estado :hover e :focus-visible (agrupados no mesmo bloco, já que o teclado deve receber o mesmo feedback visual do mouse), o background-color escurece de --mel para --mel-hover, e é adicionado um box-shadow: 0 4px 12px rgba(27, 42, 64, 0.18), criando sensação de elevação. No estado :active, acionado durante o clique, o box-shadow é removido e aplica-se transform: translateY(1px) scale(0.98), simulando o botão "afundando" fisicamente — um feedback tátil imediato de que a ação foi registrada. Já o estado :disabled neutraliza a cor de destaque, trocando o background-color para var(--linha) (neutro) e o color para var(--texto-suave), remove sombra e transformação, e aplica cursor: not-allowed, comunicando visualmente que a ação não está disponível. O .btn-secundario segue a mesma lógica de encadeamento, mas trabalha com border-color e inversão de background/color ao invés de um preenchimento sólido, já que parte de um estilo "outline". Para os formulários (contato e cadastro), a validação visual foi construída com as pseudo-classes :user-valid e :user-invalid, aplicadas a input, select e textarea. Diferentemente de :valid/:invalid (que reagem imediatamente, inclusive em campos vazios ainda não preenchidos), :user-valid/:user-invalid só são ativadas depois que o usuário efetivamente interage com o campo, evitando mostrar erro antes da hora. Quando o campo passa a conter um valor válido segundo suas regras (required, type, pattern), :user-valid aplica border-color: #2E7D32 (verde) e uma faixa lateral colorida via background-image: linear-gradient(#2E7D32, #2E7D32) com background-size: 3px 100% e background-repeat: no-repeat, criando um indicador visual lateral discreto sem precisar de ícone extra. Quando o valor é inválido após interação, :user-invalid aplica a mesma lógica com border-color: #C0392B (vermelho). Esse encadeamento garante que o próprio navegador — através da validação nativa HTML5 já configurada nos atributos required, type e pattern — dite o estado, e o CSS apenas traduza esse estado em uma resposta visual imediata e consistente, sem necessidade de scripts adicionais para essa camada específica de feedback.



9 Componentes de feedback

Evidências dos componentes visuais





Checklist de qualidade

- *Design system* consistente desenvolvido (com variáveis de cores, tipografia hierárquica e espaçamentos padronizados).
- *Layout* macro estruturado através da técnica *CSS Grid* (com *template* de 12 colunas e no mínimo 5 *breakpoints* distintos).
- Disposição e alinhamento microscópico de componentes internos aplicados e estabilizados utilizando a arquitetura *Flexbox*.
- Módulo de navegação principal funcional (com estrutura *dropdown* testada) e menu alternativo otimizado para versão *mobile* em comportamento de hambúrguer.
- Sistema de *cards* padronizado e validação visual de estados interativos integralmente aplicada nos botões e elementos de formulário.
- Componentes visuais semânticos de *feedback* incorporados ao *layout* (incluindo modais funcionais, alertas e *tags* para categorização contextual das ONGs).



Reflexão sobre a aprendizagem

Nesta segunda experiência prática, desenvolvi a camada de estilização e interatividade do projeto da ONG Amor de Patas, aplicando um sistema de design consistente, Grid de 12 colunas com breakpoints responsivos, Flexbox, navegação com dropdown e menu hambúrguer, além de componentes de feedback, como modais, alertas, badges e toasts. Entre os principais aprendizados, destaco a organização das variáveis CSS para cores, tipografia e espaçamentos, além da aplicação adequada de Grid na estrutura das páginas e Flexbox no alinhamento interno dos componentes. Como oportunidade de melhoria, percebi a importância de testar as interfaces em diferentes tamanhos de tela. Durante o desenvolvimento, identifiquei um problema no modal de detalhes dos animais, que cortava parte do conteúdo em janelas menores. A correção exigiu separar o posicionamento do modal de seu conteúdo interno e permitir a rolagem, tornando o componente mais adaptável. A experiência reforçou que o desenvolvimento de boas interfaces não depende apenas do conhecimento das propriedades CSS, mas também da integração entre organização visual, responsividade, acessibilidade e testes reais de interação. Esse aprendizado contribui para minha formação profissional e para a criação de interfaces mais consistentes e adequadas às necessidades dos usuários.

